

PYTHON PROGRAMMING

COMPLETE HANDBOOK — BASIC TO ADVANCED

PART 1

Module 1: Introduction to Python

Module 2: Python Basics

Module 3: Operators

For Students · Job Seekers · College Learners · Beginners

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
HOW TO USE THIS BOOK	4
MODULE 1: INTRODUCTION TO PYTHON	5
1.1 What is Python?	5
1.2 History of Python	5
1.3 Features of Python.....	5
1.4 Applications of Python	6
1.5 Python Installation.....	6
1.6 Python IDEs	6
1.7 Running Python Programs	7
1.8 First Python Program	7
1.9 Python Execution Process.....	7
Interview Questions — Module 1	8
MCQs — Module 1	8
Exercises — Module 1.....	9
MODULE 2: PYTHON BASICS.....	10
2.1 Keywords.....	10
2.2 Identifiers	10
2.3 Variables.....	10
2.4 Data Types.....	11
2.5 Type Conversion	11
2.6 Input and Output	12
2.7 Comments	12
2.8 Indentation	12
Programs — Module 2	13
Interview Questions — Module 2	13
MCQs — Module 2	14
Exercises — Module 2.....	14
MODULE 3: OPERATORS.....	16
3.1 Arithmetic Operators.....	16
3.2 Assignment Operators	16
3.3 Comparison Operators	16
3.4 Logical Operators	17
3.5 Bitwise Operators	17
3.6 Membership Operators	17

3.7 Identity Operators	18
3.8 Operator Precedence.....	18
Programs — Module 3	18
Interview Questions — Module 3	19
MCQs — Module 3.....	19
Exercises — Module 3.....	20
QUICK REVISION NOTES — MODULES 1-3	21
COMMON ERRORS AND FIXES — MODULES 1-3	22
CODING BEST PRACTICES.....	23

HOW TO USE THIS BOOK

This handbook is organised into Modules, each covering one topic in full: theory in simple English, worked examples with output, interview questions, MCQs, and practice exercises.

- Read the theory first, then type and run every example yourself — do not just read code.
- Attempt the exercises before checking any solution.
- Use the MCQs and interview questions for quick self-testing before exams or interviews.
- The 'Common Errors' boxes show mistakes beginners make most often — read them carefully.
- This is Part 1 of the full handbook (Modules 1-3). Further parts continue the numbering.

MODULE 1: INTRODUCTION TO PYTHON

1.1 What is Python?

Python is a high-level, general-purpose programming language created by Guido van Rossum. 'High-level' means Python code looks close to plain English rather than machine instructions, so it is easier to read, write, and maintain than languages like C. Python is interpreted, which means a program called the Python interpreter reads and runs your code line by line, instead of converting the whole program into machine code before running (as compiled languages like C++ do).

Python is used for web development, data science, artificial intelligence, automation, scripting, game development, and much more. It is one of the most popular languages in the world because it is easy to learn and extremely powerful once mastered.

Real-World Analogy

Think of Python as giving instructions to a very obedient but literal-minded assistant. If you write clear, simple English-like instructions (Python code), the assistant (interpreter) carries them out exactly, one line at a time.

1.2 History of Python

- Python was created by Guido van Rossum and released in 1991.
- The name 'Python' comes from the British comedy show 'Monty Python's Flying Circus' — not the snake.
- Python 2.0 was released in 2000, introducing list comprehensions and garbage collection.
- Python 3.0 was released in 2008 as a redesign that fixed core language flaws; it is not fully backward-compatible with Python 2.
- Python 2 officially reached end-of-life on 1 January 2020. All modern learning and development uses Python 3.

1.3 Features of Python

Operator	Definition	Syntax	Example → Output	Real-life Use
Simple & Readable	Syntax resembles plain English	<code>print("Hi")</code>	Easy for beginners	Teaching first language
Interpreted	Executes line by line, no separate compile step	<code>python file.py</code>	Instant feedback	Rapid prototyping
Free & Open Source	Free to use, download, and distribute	<code>python.org</code>	No licence cost	Startups, students
Dynamically Typed	No need to declare variable types	<code>x = 5</code>	x can later hold text	Fast scripting
Portable	Runs on Windows, macOS, Linux unchanged	Same <code>.py</code> file	Write once, run anywhere	Cross-platform tools
Extensive Libraries	Huge standard library + PyPI packages	<code>import numpy</code>	1000s of ready modules	Data science, AI, web

Operator	Definition	Syntax	Example → Output	Real-life Use
Object-Oriented	Supports classes, objects, inheritance	class Car:	Models real-world entities	Large applications

1.4 Applications of Python

- Web Development — Django, Flask (e.g., Instagram, Pinterest use Python)
- Data Science & Analytics — Pandas, NumPy
- Artificial Intelligence & Machine Learning — TensorFlow, scikit-learn, PyTorch
- Automation / Scripting — automating Excel reports, file renaming, email sending
- Game Development — Pygame
- Desktop GUI Applications — Tkinter, PyQt
- Cybersecurity & Ethical Hacking — writing scanning and testing scripts
- Embedded Systems / IoT — MicroPython on microcontrollers

1.5 Python Installation

Steps to install Python on Windows/macOS/Linux:

1. Visit the official site: python.org/downloads
2. Download the latest stable Python 3.x installer for your operating system.
3. Run the installer. On Windows, tick the checkbox 'Add Python to PATH' — this is the single most common step beginners forget.
4. Complete installation, then open a terminal/command prompt.
5. Verify installation by typing: `python --version` (or `python3 --version` on macOS/Linux).

Common Beginner Mistake

Forgetting to tick 'Add Python to PATH' during Windows installation causes the error: 'python is not recognised as an internal or external command'. Fix: re-run the installer and enable that option, or manually add Python's install folder to the PATH environment variable.

1.6 Python IDEs

An IDE (Integrated Development Environment) is software used to write, run, and debug code.

IDE	Best For	Pros	Cons
IDLE	Absolute beginners	Comes bundled with Python, very simple	Very basic, no advanced features
VS Code	General development	Lightweight, huge extension ecosystem	Needs Python extension setup
Jupyter Notebook	Data science, learning	Run code in cells, see output instantly, great for charts	Not ideal for large applications
PyCharm	Professional/large projects	Powerful debugging, refactoring tools	Heavier, Community edition has limits

IDE	Best For	Pros	Cons
Google Colab	ML/AI, no-install learning	Runs in browser, free GPU access	Needs internet connection

1.7 Running Python Programs

There are two main ways to run Python code:

- Interactive Mode: type `python` in the terminal to open the interpreter (shown as `>>>`) and run one line at a time — good for quick testing.
- Script Mode: write code in a file with a `.py` extension (e.g., `hello.py`) and run it with: `python hello.py` — used for real programs.

1.8 First Python Program

```
# hello.py
# My first Python program
print("Hello, World!")
```

Output:

Hello, World!

Line-by-line explanation:

Line	Explanation
1	A comment describing the file. Comments start with <code>#</code> and are ignored by the interpreter.
2	Another comment explaining the program's purpose.
3	<code>print()</code> is a built-in function that displays text on the screen. The text inside quotes <code>"Hello, World!"</code> is a string.

1.9 Python Execution Process

Text-based flowchart of what happens when you run a `.py` file:

```
[ Source Code (file.py) ]
    |
    v
[ Python Interpreter reads code line by line ]
    |
    v
[ Code compiled internally to Bytecode (.pyc) ]
    |
    v
[ Python Virtual Machine (PVM) executes bytecode ]
    |
    v
[ Output shown on screen / file / etc. ]
```

Note: Python is often called 'interpreted', but internally it first compiles source code to an intermediate form called bytecode, and the PVM (Python Virtual Machine) then executes that bytecode. This is why Python is sometimes described as both compiled and interpreted.

Interview Questions — Module 1

Q1. What type of language is Python — compiled or interpreted?

Ans: Python is generally called interpreted because the interpreter runs code line by line. Internally, it first compiles source code into bytecode, which the Python Virtual Machine then interprets — so it is technically a hybrid.

Q2. Who created Python and when?

Ans: Guido van Rossum created Python; it was first released in 1991.

Q3. Why is Python called a 'high-level' language?

Ans: Because its syntax is close to human (English-like) language and it hides low-level details like memory management from the programmer.

Q4. Name any four real-world applications of Python.

Ans: Web development (Django/Flask), Data Science (Pandas/NumPy), AI/ML (TensorFlow), and Automation/Scripting.

Q5. What is the difference between Python 2 and Python 3?

Ans: Python 3 is a redesigned, non-backward-compatible version with better Unicode support, `print()` as a function (not a statement), and true division by default. Python 2 reached end-of-life in January 2020.

Q6. What is PVM?

Ans: PVM (Python Virtual Machine) is the runtime engine that executes Python bytecode produced by the interpreter.

Q7. Which IDE would you recommend for a complete beginner and why?

Ans: IDLE, because it comes bundled with the Python installer and has a minimal, distraction-free interface suited for learning the basics.

MCQs — Module 1

No.	Question	Answer	Options (A-D)
1	Who is the creator of Python?	B	A) Dennis Ritchie B) Guido van Rossum C) James Gosling D) Bjarne Stroustrup
2	Python is named after:	C	A) A snake B) Its founder's pet C) A comedy show D) A Greek god
3	Which file extension is used for Python scripts?	B	A) .pt B) .py C) .pyt D) .python
4	Which of the following runs Python code interactively?	A	A) Python shell (>>>) B) hello.py only C) Notepad D) Web browser
5	Python 2 reached end-of-life in:	D	A) 2015 B) 2018 C) 2019 D) 2020
6	Which is NOT a Python IDE?	D	A) PyCharm B) IDLE C) VS Code D) XAMPP

No.	Question	Answer	Options (A-D)
7	<code>print("Hello")</code> will display:	A	A) Hello B) "Hello" C) <code>print(Hello)</code> D) Error
8	Which company/organisation manages Python's official releases?	B	A) Microsoft B) Python Software Foundation C) Oracle D) Google
9	What must you tick during Windows installation to use Python from any folder?	C	A) Install pip B) Install for all users C) Add Python to PATH D) Install documentation
10	The Python interpreter converts source code into:	B	A) Machine code directly B) Bytecode C) Assembly D) HTML

Exercises — Module 1

1. Install Python on your computer and verify the version using the terminal.
2. Write and run a program that prints your name, age, and city on three separate lines.
3. Open the Python interactive shell and use it as a calculator to compute $25 * 4 + 10$.
4. List three applications where Python is used in the industry you want to work in.
5. Explain in your own words the difference between interactive mode and script mode.

MODULE 2: PYTHON BASICS

2.1 Keywords

Keywords are reserved words in Python that have a special, fixed meaning. You cannot use them as variable names. Python 3 has 35 keywords:

```
False  None  True  and  as  assert  async  await
break  class  continue  def  del  elif  else  except
finally  for  from  global  if  import  in  is
lambda  nonlocal  not  or  pass  raise  return  try
while  with  yield
```

You can view this list anytime with:

```
import keyword
print(keyword.kwlist)
```

2.2 Identifiers

An identifier is the name given to variables, functions, classes, etc. Rules for valid identifiers:

- Can contain letters, digits, and underscore (_) only.
- Cannot start with a digit (1name is invalid, name1 is valid).
- Cannot be a Python keyword (class, for, etc. are not allowed).
- Case-sensitive: Age and age are different identifiers.
- Cannot contain spaces or special symbols like @, \$, %.

Identifier	Valid?	Reason
student_name	Valid	Letters and underscore only
2ndYear	Invalid	Starts with a digit
total\$	Invalid	Contains special symbol \$
class	Invalid	It is a reserved keyword

2.3 Variables

A variable is a name that refers to a value stored in memory. In Python, you do not declare a type — the type is decided automatically based on the value assigned.

```
name = "Riya"      # string
age = 21           # integer
height = 5.6       # float
a, b, c = 1, 2, 3  # multiple assignment
x = y = 100        # same value to multiple variables
```

Real-World Analogy

A variable is like a labelled box. The label (variable name) stays the same, but you can change what is inside the box (the value) at any time.

2.4 Data Types

Data Type	Example	Description
int	x = 10	Whole numbers, positive or negative
float	x = 10.5	Decimal / floating-point numbers
complex	x = 2 + 3j	Numbers with a real and imaginary part
str	x = "Hi"	Sequence of characters (text)
bool	x = True	Represents True or False
list	x = [1,2,3]	Ordered, changeable collection
tuple	x = (1,2,3)	Ordered, unchangeable collection
set	x = {1,2,3}	Unordered collection of unique items
dict	x = {"a":1}	Key-value pairs
NoneType	x = None	Represents 'no value'

Use `type()` to check the data type of any variable:

```
x = 10
print(type(x))
```

Output:

```
<class 'int'>
```

2.5 Type Conversion

Type conversion means changing a value from one data type to another. There are two kinds:

- **Implicit Conversion:** Python does it automatically (e.g., `int + float = float`).
- **Explicit Conversion (Type Casting):** the programmer converts manually using `int()`, `float()`, `str()`, etc.

```
a = 5          # int
b = 2.5        # float
c = a + b      # implicit: a is converted to float automatically
print(c)       # 7.5

num_str = "100"
num_int = int(num_str) # explicit conversion
print(num_int + 50)    # 150
```

Output:

```
7.5
150
```

2.6 Input and Output

`input()` reads text typed by the user (always returns a string). `print()` displays output on the screen.

```
name = input("Enter your name: ")
age = int(input("Enter your age: ")) # convert string to int
print(f"Hello {name}, you are {age} years old.")
```

Output:

```
Enter your name: Aman
Enter your age: 20
Hello Aman, you are 20 years old.
```

Common Beginner Mistake

`input()` always returns a string, even if the user types a number. Forgetting to convert it with `int()` or `float()` before doing maths causes a `TypeError: unsupported operand type(s)`.

2.7 Comments

- Single-line comment: starts with `#` — everything after it on that line is ignored.
- Multi-line comment: Python has no true multi-line comment syntax; programmers use a triple-quoted string as a workaround, or `#` on every line.
- Docstring: a triple-quoted string right after a function/class definition, used to document what it does.

```
# This is a single-line comment
"""
This looks like a multi-line comment
but is technically a string that is not assigned to anything.
"""

def add(a, b):
    """Returns the sum of a and b."""
    return a + b
```

2.8 Indentation

Indentation (spaces at the start of a line) is not optional style in Python — it defines code blocks, replacing the `{ }` braces used in languages like C or Java. Standard practice is 4 spaces per indentation level.

```
if 5 > 2:
    print("5 is greater") # indented — inside the if block
    print("Still inside")
print("Outside the if block") # not indented
```

Common Beginner Mistake

Mixing tabs and spaces, or inconsistent indentation, causes: `IndentationError: unexpected indent (or unindent)`. Fix: use a code editor that shows whitespace, and stick to 4 spaces consistently — never mix with tabs.

Programs — Module 2**Program 1: Simple Interest Calculator**

```
principal = float(input("Enter principal amount: "))
rate = float(input("Enter rate of interest: "))
time = float(input("Enter time in years: "))

simple_interest = (principal * rate * time) / 100
print("Simple Interest =", simple_interest)
```

Line	Explanation
1-3	Take three inputs from the user and convert each to float since <code>input()</code> returns strings.
5	Apply the simple interest formula: $(P \times R \times T) / 100$.
6	Print the result using <code>print()</code> with multiple comma-separated arguments.

Output:

```
Enter principal amount: 10000
Enter rate of interest: 5
Enter time in years: 2
Simple Interest = 1000.0
```

Program 2: Swap Two Numbers Without a Third Variable

```
a, b = 5, 10
a, b = b, a
print("a =", a, " b =", b)
```

Output:

```
a = 10  b = 5
```

Interview Questions — Module 2**Q1. What is the difference between a keyword and an identifier?**

Ans: A keyword is a reserved word with a fixed meaning in Python (e.g., `if`, `for`) that cannot be used as a name. An identifier is a name chosen by the programmer for a variable, function, or class.

Q2. Is Python statically or dynamically typed?

Ans: Dynamically typed — you do not declare a variable's type; it is decided at runtime based on the assigned value, and can even change later.

Q3. What does `input()` always return?

Ans: A string (str), regardless of what the user types — numbers must be explicitly converted using `int()` or `float()`.

Q4. What is the difference between implicit and explicit type conversion?

Ans: Implicit conversion is done automatically by Python (e.g., int to float in mixed arithmetic). Explicit conversion is done manually by the programmer using functions like `int()`, `float()`, `str()`.

Q5. Why is indentation important in Python?

Ans: Python uses indentation instead of braces to define blocks of code (like if, for, function bodies). Incorrect indentation causes an `IndentationError` or changes program logic.

Q6. What is a docstring?

Ans: A triple-quoted string placed as the first statement in a function, class, or module, used to describe what it does; accessible via the `__doc__` attribute.

Q7. Name any five Python data types.

Ans: int, float, str, bool, list (also complex, tuple, set, dict, `NoneType`).

Q8. Can a variable name start with a number?

Ans: No. A variable/identifier must start with a letter or an underscore, never a digit.

MCQs — Module 2

No.	Question	Answer	Options (A-D)
1	Which of these is a valid Python identifier?	A	A) <code>_value</code> B) <code>2value</code> C) <code>my-value</code> D) <code>class</code>
2	<code>input()</code> function returns data of type:	C	A) int B) float C) str D) bool
3	How many spaces are conventionally used for one indentation level?	B	A) 2 B) 4 C) 8 D) 1
4	Which is NOT a Python keyword?	D	A) <code>pass</code> B) <code>yield</code> C) <code>lambda</code> D) <code>print</code>
5	<code>type(5.0)</code> returns:	B	A) int B) float C) str D) complex
6	Which symbol starts a single-line comment?	A	A) <code>#</code> B) <code>//</code> C) <code>/*</code> D) <code>--</code>
7	<code>x = y = 10</code> means:	C	A) Syntax Error B) Only <code>x = 10</code> C) Both <code>x</code> and <code>y</code> are 10 D) Only <code>y = 10</code>
8	<code>int("25") + 5</code> evaluates to:	B	A) "255" B) 30 C) Error D) 25.5
9	Which data type stores True/False?	D	A) int B) str C) list D) bool
10	What does a triple-quoted string right after a function definition act as?	A	A) Docstring B) Comment (ignored) C) Error D) Variable

Exercises — Module 2

- Write a program to swap two numbers using a temporary variable, and again without one.
- Take a user's name and age as input and print a formatted sentence using an f-string.
- Write a program that checks and prints the data type of five different variables.
- Convert the string "3.14" to a float and add 1 to it; print the result and its type.
- Identify which of these are valid identifiers: `1data`, `data_1`, `data one`, `_data`, if

6. Write a program to calculate the area of a rectangle by taking length and width as input.
7. Explain, with a code example, why indentation errors occur.
8. Write a docstring for a function named `greet` that prints 'Hello'.

MODULE 3: OPERATORS

Operators are special symbols that perform operations on values (called operands). Python groups operators into seven categories.

3.1 Arithmetic Operators

Operator	Definition	Syntax	Example → Output	Real-life Use
+	Addition	$a + b$	$5 + 2 \rightarrow 7$	Total bill amount
-	Subtraction	$a - b$	$5 - 2 \rightarrow 3$	Remaining balance
*	Multiplication	$a * b$	$5 * 2 \rightarrow 10$	Price × quantity
/	Division (true, float result)	a / b	$5 / 2 \rightarrow 2.5$	Average calculation
//	Floor Division (drops decimal)	$a // b$	$5 // 2 \rightarrow 2$	Items per box (whole)
%	Modulus (remainder)	$a \% b$	$5 \% 2 \rightarrow 1$	Check odd/even
**	Exponent (power)	$a ** b$	$5 ** 2 \rightarrow 25$	Compound interest

3.2 Assignment Operators

Operator	Definition	Syntax	Example → Output	Real-life Use
=	Assigns value	$x = 5$	$x \rightarrow 5$	Storing a score
+=	Add and assign	$x += 3$	$x \rightarrow x+3$	Adding to a running total
-=	Subtract and assign	$x -= 3$	$x \rightarrow x-3$	Reducing stock count
*=	Multiply and assign	$x *= 3$	$x \rightarrow x*3$	Applying a multiplier
/=	Divide and assign	$x /= 3$	$x \rightarrow x/3$	Splitting a bill
%=	Modulus and assign	$x \% = 3$	$x \rightarrow x\%3$	Cyclic counters
=	Power and assign	$x ** = 3$	$x \rightarrow x3$	Growth calculations

3.3 Comparison Operators

Operator	Definition	Syntax	Example → Output	Real-life Use
==	Equal to	$a == b$	$5 == 5 \rightarrow \text{True}$	Password match check
!=	Not equal to	$a != b$	$5 != 3 \rightarrow \text{True}$	Detecting a change
>	Greater than	$a > b$	$5 > 3 \rightarrow \text{True}$	Eligibility check (age)

Operator	Definition	Syntax	Example → Output	Real-life Use
<	Less than	<code>a < b</code>	<code>5 < 3 → False</code>	Stock below threshold
>=	Greater or equal	<code>a >= b</code>	<code>5 >= 5 → True</code>	Minimum marks check
<=	Less or equal	<code>a <= b</code>	<code>5 <= 5 → True</code>	Budget limit check

3.4 Logical Operators

Operator	Definition	Syntax	Example → Output	Real-life Use
and	True if both conditions are True	<code>a and b</code>	<code>(5>2) and (3>1) → True</code>	Login: username AND password correct
or	True if at least one condition is True	<code>a or b</code>	<code>(5>2) or (3<1) → True</code>	Discount if student OR senior citizen
not	Reverses the boolean result	<code>not a</code>	<code>not(5>2) → False</code>	Access denied if NOT logged in

3.5 Bitwise Operators

Bitwise operators work directly on the binary representation of integers.

Operator	Definition	Syntax	Example → Output	Real-life Use
&	Bitwise AND	<code>a & b</code>	<code>5 & 3 → 1</code>	Permission flag checks
 	Bitwise OR	<code>a b</code>	<code>5 3 → 7</code>	Combining permission flags
^	Bitwise XOR	<code>a ^ b</code>	<code>5 ^ 3 → 6</code>	Simple encryption logic
~	Bitwise NOT	<code>~a</code>	<code>~5 → -6</code>	Low-level bit manipulation
<<	Left shift	<code>a << 1</code>	<code>5 << 1 → 10</code>	Fast multiply by 2
>>	Right shift	<code>a >> 1</code>	<code>5 >> 1 → 2</code>	Fast divide by 2

3.6 Membership Operators

Operator	Definition	Syntax	Example → Output	Real-life Use
in	True if value found in sequence	<code>x in seq</code>	<code>'a' in 'cat' → True</code>	Check if item is in a cart list
not in	True if value NOT found in sequence	<code>x not in seq</code>	<code>'z' not in 'cat' → True</code>	Check username not already taken

3.7 Identity Operators

Operator	Definition	Syntax	Example → Output	Real-life Use
is	True if both refer to the same object in memory	a is b	a is b	Checking if x is None
is not	True if they do NOT refer to the same object	a is not b	a is not b	Validating object identity

is vs == — a Frequent Confusion

`==` compares VALUES (are the contents equal?). `'is'` compares IDENTITY (are they literally the same object in memory?). Two different lists with the same items are `== True` but `'is' False`.

3.8 Operator Precedence

When an expression has multiple operators, Python evaluates them in this order (highest to lowest):

1. `**` (Exponent)
2. `+ x` , `- x` , `~x` (Unary plus/minus/NOT)
3. `*` / `/` / `//` / `%` (Multiplication, Division, Floor Div, Modulus)
4. `+` / `-` (Addition, Subtraction)
5. `<<` / `>>` (Bitwise shifts)
6. `&` (Bitwise AND)
7. `^` / `|` (Bitwise XOR, OR)
8. `==` / `!=` / `>` / `<` / `>=` / `<=` (Comparisons)
9. `not` (Logical NOT)
10. `and` (Logical AND)
11. `or` (Logical OR)

Example:

```
result = 10 + 2 * 3    # * evaluated before +
print(result)         # 16, not 36
```

Output:

16

Programs — Module 3

Program 1: Check Odd or Even Using Modulus

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print(num, "is Even")
else:
    print(num, "is Odd")
```

Output:

```
Enter a number: 7
7 is Odd
```

Program 2: Simple Login Check Using Logical Operators

```
username = input("Username: ")
password = input("Password: ")

if username == "admin" and password == "1234":
    print("Login Successful")
else:
    print("Login Failed")
```

Line	Explanation
1-2	Take username and password as string input.
4	and requires BOTH conditions to be True — correct username AND correct password.
5-6	Print success or failure based on the combined result.

Interview Questions — Module 3**Q1. What is the difference between / and // in Python?**

Ans: / performs true division and always returns a float (e.g., $7/2 = 3.5$). // performs floor division and returns the largest integer not greater than the result, discarding the remainder ($7//2 = 3$).

Q2. What is the difference between == and is?

Ans: == compares whether two values are equal. 'is' checks whether two variables point to the exact same object in memory (identity), not just equal values.

Q3. What does the modulus operator (%) return?

Ans: The remainder after division. It is commonly used to check if a number is even/odd ($\text{num} \% 2 == 0$) or to cycle through a fixed range of values.

Q4. Explain operator precedence with an example.

Ans: It is the order in which operators are evaluated when an expression has more than one. E.g. in $10 + 2 * 3$, * has higher precedence than +, so multiplication happens first, giving 16, not 36.

Q5. What is the use of the 'in' operator?

Ans: It checks membership — whether a value exists within a sequence such as a string, list, or tuple, returning True or False.

Q6. Give one real-life use of the 'and' logical operator.

Ans: A login system that grants access only when BOTH the username AND password entered are correct.

MCQs — Module 3

No.	Question	Answer	Options (A-D)
1	10 // 3 evaluates to:	B	A) 3.33 B) 3 C) 1 D) 4
2	10 % 3 evaluates to:	A	A) 1 B) 3 C) 0 D) 10

No.	Question	Answer	Options (A-D)
3	Which operator checks value equality?	C	A) = B) is C) == D) equals
4	2 ** 3 evaluates to:	D	A) 6 B) 5 C) 9 D) 8
5	Which operator combines two conditions, requiring BOTH true?	B	A) or B) and C) not D) is
6	'py' in 'python' evaluates to:	A	A) True B) False C) Error D) None
7	Which has the highest precedence?	C	A) + B) == C) ** D) or
8	x = 5; x += 2 makes x:	B	A) 5 B) 7 C) 2 D) Error
9	Bitwise AND operator symbol is:	A	A) & B) && C) and D)
10	a is b checks:	D	A) Value equality B) Data type C) Length D) Same object identity

Exercises — Module 3

1. Write a program to calculate the area and perimeter of a circle using arithmetic operators.
2. Write a program that checks whether a number is divisible by both 3 and 5 using logical operators.
3. Predict the output of: `print(2 + 3 * 4 ** 2)` and explain the precedence used.
4. Write a program demonstrating the difference between `==` and `is` using two equal lists.
5. Use the `'in'` operator to check whether a given letter exists in a word entered by the user.
6. Write a program using bitwise left shift (`<<`) to multiply a number by 4.

QUICK REVISION NOTES — MODULES 1-3

- Python is a high-level, interpreted, dynamically-typed, object-oriented language created by Guido van Rossum (1991).
- Run code two ways: interactive shell (`>>>`) for quick tests, or script mode (`python file.py`) for real programs.
- 35 keywords are reserved; identifiers must start with a letter/underscore, contain only letters/digits/underscore, and are case-sensitive.
- Python decides a variable's type automatically from its value — no type declaration needed.
- Core data types: `int`, `float`, `complex`, `str`, `bool`, `list`, `tuple`, `set`, `dict`, `NoneType`.
- `input()` always returns a string — convert with `int()`/`float()` before doing maths.
- Indentation (4 spaces) defines code blocks — it is mandatory, not stylistic.
- Arithmetic: `+` `-` `*` `/` `//` `%` `**`. Remember: `/` gives float, `//` gives integer (floor) result.
- `==` compares values; `'is'` compares object identity.
- Operator precedence (high→low, key ones): `**` → unary `+`/`-` → `*` `/` `//` `%` → `+` `-` → comparisons → `not` → `and` → `or`.

COMMON ERRORS AND FIXES — MODULES 1-3

Error	Cause	Fix
'python' is not recognised	Python not added to PATH	Reinstall and tick 'Add to PATH'
IndentationError	Inconsistent spaces/tabs	Use 4 spaces consistently
TypeError: unsupported operand	Adding str + int without conversion	Convert with int()/float()/str()
SyntaxError: invalid syntax near ':'	Missing colon after if/for/def	Add ':' at end of the header line
NameError: name not defined	Using a variable before assigning it	Assign a value before using it

CODING BEST PRACTICES

- Use meaningful variable names: `total_price`, not `tp` or `x`.
- Follow PEP 8 style guide: 4-space indentation, lowercase_`_with_underscores` for variables/functions.
- Add comments to explain WHY, not WHAT (the code already shows what).
- Keep lines under ~79-99 characters for readability.
- Test small pieces of code in the shell before combining into a full program.
- Always convert `input()` before using it in arithmetic.